

Cloud Accounts & Deployment

Moving the pipeline off your laptop: a DigitalOcean server for compute, Azure object storage for the data lake.

Covers: why this shape · the server · Azure storage · migration · security · costs | August 2026

1. The shape, and why

Two cloud pieces, each doing exactly one job. A rented Linux server — a DigitalOcean droplet in their New York datacenter — runs the containers: the same compose file you have been using, nothing changes but the host. An Azure storage account holds what must outlive any machine: the immutable Parquet archive of raw aircraft states, and nightly database backups.

The division is deliberate, and it is the pattern worth being able to defend: **compute is disposable, storage is not**. The droplet can die, be rebuilt, or be resized and the system loses nothing that matters — the archive and backups live in object storage, decoupled from any host. Meanwhile the warehouse and stream processing stay self-hosted, because at this scale a managed warehouse would be cost without engineering content. Storage is the piece where cloud genuinely earns its place: eleven nines of durability for about a dime a month.

The money, plainly: the server is the project's one real line item — **\$48 a month, about \$96 for the build-and-soak window**. New-account credits are token-sized these days and change without notice; budget on the list price and treat any credit as a pleasant rounding error. The Azure storage rounds to zero on a new account's free tier and to pennies after. A billing alert (step A2) and a firm teardown date at the end are the two cost controls that actually work.

2. Part A — the server

A1. Sizing: why 8 GB and not 4

The full stack peaks near 7 GB once stream processing arrives — the processing engine's driver alone is configured at 4 GB. The tempting 4 GB tier will run today's ingestion and then fail, confusingly, one phase later. Pick the right droplet once:

DROPLET	SPEC	PRICE	VERDICT
Basic 4 GB	2 vCPU · 4 GB	\$24/mo	Runs ingestion only — a valid <i>starter</i> tier if you resize before the processing phase (below)
Basic Regular 8 GB	4 vCPU · 8 GB · 160 GB disk	\$48/mo	The one to create — sized for the whole pipeline, not just today's phase
Basic Premium 8 GB	4 vCPU · 8 GB, newer CPUs + NVMe	\$56/mo	Unnecessary — this workload is I/O-light and never CPU-bound

The budget-conscious schedule, and the one this deployment uses: **start on 4 GB, resize to 8 GB the day stream processing deploys**. A droplet resizes in place — power off for about a minute, pick the larger plan, power on; the disk and every byte on it are untouched, and the containers resume on boot. Put the resize on the same calendar line as the processing phase's first deploy; the failure mode of forgetting is an out-of-memory crash loop that looks like a software bug and wastes an evening.

A2. Create the account and the droplet (~15 min)

1. Sign up at `digitalocean.com`. Card required; billing is hourly against the monthly cap, so a droplet destroyed mid-month costs only the hours it lived.
2. **Set the billing alert before creating anything.** Left nav → *Settings* → *Billing* → set an alert at **\$50**. One droplet should never quite reach it in a month — the alert firing early means something extra is running that you forgot about. Put the teardown date in your calendar now: the day the analysis ships, the droplet dies.

3. Add your deploy key: *Settings* → *Security* → *Add SSH Key*. Paste the *public* half — it was generated for you at `~/ .ssh/skynyc_vps_ed25519.pub` ; print it with:

```
cat ~/ .ssh/skynyc_vps_ed25519.pub
```

4. Create the server: **Create** → **Droplets** → region **New York (NYC3)** → image **Ubuntu 24.04 LTS** → plan **Basic** → **Regular** → **8 GB / 4 vCPU (\$48)** → authentication: **SSH Key**, tick the key from step 3 → hostname `skynyc-1` → Create Droplet. It boots in under a minute.

5. Copy the droplet's public IP into your `.env` :

```
VPS_HOST=root@<the-ip>
```

PASSWORDS ARE NOT PART OF THIS SETUP

The server should only ever be reachable by SSH key — that is why the key went in before the droplet was created. If you skipped step 3, do not fall back to a root password; add the key and rebuild the droplet. It costs nothing and takes a minute.

A3. Deploy (~5 min, one command)

From the repository on your laptop:

```
./scripts/deploy.sh
```

On a fresh server this installs Docker, adds a 4 GB swap file as headroom (droplets ship with none), syncs the project (including `.env` — the server needs the credentials to poll), starts the infrastructure, creates the topics, and starts ingestion. It finishes by printing service status and the latest poll line. Re-running it later is the update path: it syncs changes and restarts what moved.

Then stop the laptop's copy — one collector, one credit budget:

```
make down          # on the laptop; volumes stay, nothing is lost
```

A4. Lock the door (~5 min)

First, understand where the real defense lives. A common and dangerous assumption is that a host firewall protects containers — it does not: Docker programs its own rules ahead of the firewall's, so a container port published as `9092:9092` is open to the internet no matter what `ufw` says. This project closes that hole in the compose file itself: every published port is bound to `127.0.0.1`, so Kafka, Postgres, and Grafana are reachable only from inside the server. That protection deploys with the code and cannot be forgotten during a rebuild.

With that in place, the host hardening is three commands, once:

```
ssh -i ~/ .ssh/skynyc_vps_ed25519 root@<the-ip>

# 1. firewall for the host itself: SSH in, nothing else
ufw default deny incoming && ufw default allow outgoing && ufw allow OpenSSH && ufw enable

# 2. no password logins, ever
sed -i 's/^#\?PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/sshd_config
systemctl restart ssh

# 3. unattended security patches
apt-get install -y unattended-upgrades
```

When you want the dashboard, open a tunnel from the laptop and browse `localhost:3000` as usual — the tunnel terminates inside the server, which is exactly where the loopback-bound ports listen:

```
ssh -i ~/ .ssh/skynyc_vps_ed25519 -N -L 3000:localhost:3000 root@<the-ip>
```

3. Part B — Azure storage

B1. What it is for

One storage account, two containers. `bronze` receives the Parquet archive the stream-processing phase writes — the system of record, since the aircraft API cannot serve history. `backups` receives a nightly database dump. The account is created as StorageV2 with the hierarchical namespace enabled — that is what makes it a data lake (ADLS Gen2) rather than a plain blob bucket, and it is what analytics engines expect to write to.

B2. Create the account (~10 min)

1. Sign up at `azure.microsoft.com` — choose the free account. Card required; new accounts get 12 months of 5 GB free locally-redundant storage, which covers this project's entire footprint.
2. Pick your globally unique storage-account name — 3–24 lowercase letters and digits, e.g. `skynycbronze` plus your initials — and put it in `.env` as `AZURE_STORAGE_ACCOUNT`.
3. Sign the command line in (a browser window opens; the CLI is already installed):

```
az login
```

4. Provision everything in one shot:

```
./scripts/provision_azure.sh
```

This creates the resource group, the storage account (LRS, TLS 1.2 minimum, public blob access disabled), both containers, and a lifecycle rule that moves archive files to the cheaper Cool tier after 30 days.

5. The script's last line prints the command that reveals the access key. Run it, paste the value into `.env` as `AZURE_STORAGE_KEY`, and treat it exactly like the other secrets in that file: never committed, never pasted anywhere else, rotated from the portal if it ever leaks.

NOTHING TO BABYSIT

The storage account sits idle until the stream-processing phase starts writing to it. Creating it now means the account-verification and free-tier setup are behind you before they can block a build session.

4. What moved, what didn't

PIECE	BEFORE	NOW
Producers, consumer, Kafka, Postgres, Grafana	Laptop containers	Same containers, droplet
Raw archive (from the processing phase)	Laptop disk was the plan	Azure <code>bronze</code> container
Database backups	Manual, local	Nightly to Azure <code>backups</code>
Laptop's job	Run everything, never sleep	Development and dashboards over a tunnel; sleep all it wants
Kafka data, Postgres data	Stay on the droplet's disk — they are the working set, not the archive; the archive and backups are what protect them	

5. Operating notes for the new shape

- **All routine checks from the operations guide run unchanged** — SSH in and use the same `make` targets, or run them through the tunnel.
- **Deploys are one command:** edit on the laptop, `./scripts/deploy.sh`. The server is not a place where code is edited.

- **The migration loses nothing that matters.** The few hours of locally collected messages predate the archive layer; the fresh droplet collection simply starts the real record. If you ever rebuild the droplet itself: provision, deploy, restore the latest dump from the backups container.
- **Respect the meter.** \$48 a month buys exactly one thing: the pipeline running without your laptop. The moment the project no longer needs it, destroy it — hourly billing stops that hour, and the archive and backups in Azure are what make that destruction free of regret.

6. The money

ITEM	MONTHLY	PROJECT (~2 MONTHS)
DigitalOcean droplet (Basic 8 GB, NYC3)	\$48	~\$96 — destroy it at project end; hourly billing stops that hour
Azure storage (free tier → pennies)	~\$0–0.15	~\$0
Everything else (APIs, software, CI)	\$0	\$0
Total	~\$48	~\$96 — the teardown date is the real budget control

When the project wraps: export the final artifacts, destroy the droplet, and the storage account keeps the archive for pennies — or download it and delete the resource group for a true zero.